Politecnico di Torino

Politecnico di Torino

Master's Degree Course in Cybersecurity

Academic Year 2025/2026

Graduation Session October 2026

Design of an Agent-Based Framework for Vulnerability Detection and Classification in Open-Source Software Repositories

Supervisors:

Matteo Boffa

Idilio Drago

Candidate:

Alessandro Medvescek

Abstract

Open source software is the foundation of modern computing, and almost every codebase in use today depends on it. The way vulnerabilities are handled in this ecosystem creates a structural problem. Under the Coordinated Vulnerability Disclosure model a flaw is corrected before it is announced, and the fixing commit on GitHub is deliberately kept quiet, carrying no reference to security. The correction is therefore public and readable in the repository, while the information needed to recognise it as a security fix is not.

This opens a window in which the projects that depend on that code are exposed: the fix cannot be installed until a new release is published, and nothing notifies users until an advisory is issued. Meanwhile, the change itself describes the vulnerability, showing which lines were wrong and how they were repaired, so that anyone who reads it can work backwards to the flaw. Reading unfamiliar code and judging its security has always required expertise and time but general purpose language models may be lowering both barriers.

Investigating this question requires a collection of repositories whose vulnerabilities are known, and assembling one is far from simple. Public vulnerability records rarely point to the commit that resolves them, and when they do the reference is often broken, while the fixing commits cannot be found by searching either, if they are silent patch, then there is no sign of what it fixes.

Earlier approaches applied deep learning to source code, while more recent ones turn language models into agents that explore a repository by themselves, and some of these systems have already found real vulnerabilities. The systems that succeed, however, rely on specialised infrastructure, while a model operating on its own performs considerably worse. This thesis examines the intermediate configuration, which reflects the resources realistically available to an ordinary user.

This thesis proposes an architecture based on AI agents that inspect a repository on their own, decide whether it contains a vulnerability and classify it with the corresponding CWE. Each agent runs inside a disposable container, with read-only access to the code and no access to the web. Three configurations are compared, differing only in what the agent is allowed to access: the code changed by a commit together with its diff file, those same changes plus the whole repository, and the repository alone with no indication that a fix exists, which reproduces the search for an unknown vulnerability. The dataset is built from repositories that still contained the vulnerability, one commit before the fix, and is restricted to recent disclosures so that the answers cannot simply be retrived from training data.

As a secondary contribution, the thesis prepares the basis for a real-time extension. A monitoring tool selects the repositories that are worth tracking, recording

which of them end up with a published vulnerability and how long after the selection. This is the component on which a continuous version of the analysis would be built, moving it from vulnerabilities that are already known to code that has not yet been the subject of any report.

Table of Contents

List of Tables II

List of Figures III

1 Introduction 1
- 1.1 Context & Motivation 1
- 1.2 Methodology 3
- 1.3 Thesis Organization 4

Bibliography 6

List of Tables

List of Figures

Chapter 1

Introduction

1.1 Context & Motivation

Open-source software is the foundation of modern computing. Individual users and companies alike rely on openly available source code, maintained by communities of volunteers and by companies alike. Open-source has become effectively universal: about 98% of the audited codebases contain open source components [1].

Open-source security is a double-edged sword: while extreme transparency enables faster bug detection and patching by the community, it also provides attackers with everything they needed to hunt for vulnerabilities. In addition, Open-source dependencies are nested in the majority of software projects, leading to possible supply-chain attacks.

The most important and known service in term of OSS[1] is GitHub: a global platform for software development based on version control and collaboration. It hosts the vast majority of the open source software in use today, with more than two hundred million active repositories [2]. At the base of the platform there is the concept of commit: a snapshot of the source code at a specific point of time. The direct consequence of the structure of this framework is the possibility of inspecting any change made to the repository as soon as it is published. The nature of the commit can be various, from bug correction to code refactoring. For the scope of this thesis, the attention will be posed on secuirity updates.

Security fixes, however, do not reach the public in the same way as ordinary changes. Under the CVD[2] model, a vulnerability is corrected before it is announced, so that a remedy already exists by the time users learn that they are at risk. During this embargo period, between the correction and the disclousure of a vulnerability,

[1]OSS stands for open-source software.

[2]CVD stands for Coordinated Vulnerability Disclosure.

the fix is committed like any other change, but the commit message is kept vague or generic, so that the change does not attract the attention of anyone watching the repository [3].

Between the moment a fix is written and the moment it actually protects there are two distinct delays [4]. The first one lies between the commit and the release: the correction is already in the repository, but it only becomes installable when the maintainers publish a new version of the package. The second one lies between that release and the publication of a security advisory, which is what tells the automated tools used by client projects that they should update. In both cases the code of the fix is already public, while the projects that depend on it are still exposed: during the first delay users cannot update at all, because no release exists yet, and during the second one they could, but nothing tells them that they should.

This practice is known as silent patch: the vulnerability is fixed, but nothing in the commit says that a vulnerability was ever there. A direct consequence is that users keep running vulnerable software, because they receive no signal telling them that a specific update matters for the security.

Outdated code is the norm rather than the exception: one study of more than 4,600 GitHub projects found that 81.5% of them kept dependencies that were no longer up to date [2], and about 92% of the audited codebases contain components that are ten or more versions behind the current release [1].

Keeping the commit message quiet, however, does not hide the vulnerability, because the information that describes it is the change itself. A silent patch shows which lines were wrong and how they were repaired, so anyone who reads it can work backwards from the correction to the flaw corrected. In addition, the openness of the code allows to search for weaknesses that nobody has reported and nobody has fixed even without the presence of a security patch.

Artificial intelligence is a relatively new technology that is developing quickly, and it has deeply changed cybersecurity, creating new scenarios, new kinds of attack, new ways of defending against them, and new ways to analyse code in search of vulnerabilities.

The analysis of unfamiliar code to find security problems is an expensive activity in terms of time and knowledge required. As a consequence, the possibility for any user to use AI general purpose models could be a game-changer.

Measuring how well an automated analysis performs on this task is harder than it seems, and the first obstacle is the data. What is needed is a set of vulnerabilities paired with the commit that fixes each of them, but this pairing is rarely available: an empirical study of 193,448 [3] entries found that only 30.99% of them include an explicit link to a patch commit, and that 41.34% of those links no longer

[3]CVE stands for Common Vulnerabilities and Exposures

work, because repositories get deleted, branches are renamed or pull requests are merged [5]. Furthermore, in many cases, commits do not provide any help to the data gathering phase since we are talking about silent patches. In addition, vulnerabilities often emerge through chains of calls that cross several functions rather than within isolated ones [6], and the fixes that touch more than one file are exactly the ones where approaches working at function level struggle. Finally, saying that a vulnerability exists is not enough, because it also has to be classified with the corresponding CWE[4], chosen among hundreds of categories that could differ from one another by subtle distinctions.

The problem is not new, and neither are the attempts to automate it. A first generation of work applied deep learning to source code in order to detect vulnerabilities [7], but a later evaluation, under realistic conditions, found that the performance of such models dropped [8]. A parallel line of research addressed the narrower task of recognising silent fixes among ordinary commits [3].

The arrival of Large Language Models reopened the question. When each vulnerable function is paired with its corrected version, models (used on their own) label almost everything as vulnerable, while agents that explore the repository by themselves are better at discriminating between the two versions [6]. Agents that are also allowed to compile, execute and fuzz the code have already found real vulnerabilities in open source projects [9], and yet, when they are asked to find and fix a vulnerability that is genuinely new, even agents are not able to solve it on their own [10].

1.2 Methodology

This work aims to answer three questions that follow from the situation described above. Firstly, whether an agent running on a general purpose model is able to identify and classify the vulnerability when it is given only the code before and after the commit and the diff file containing the changes. Furthermore, how this ability to detect and classify the vulnerability changes when more information is added to the context. Finally, whether the agent is able to detect and classify a vulnerability with access to the repository alone and no further information, which simulates the search for a zero-day vulnerability.

The proposal is an architecture based on AI agents, evaluated in three versions that differ only in what the agent is allowed to see:

- the agent receives only the code changed by a commit, in its version before and after the correction together with the corresponding diff;

[4]CWE stands for Common Weakness Enumeration

- it receives the whole repository together with those same changes;
- it receives only the repository, with no indication that a fix exists or where to look.

Each version is asked for the same answer: whether a vulnerability is present and which CWE it corresponds to, so that the three results can be compared directly.

As a secondary contribution, the thesis also lays the groundwork for a real time implementation. The evaluation above needs a vulnerability that is already known, which is what makes the measurement possible but also confines it to the past. The tool developed for this purpose works in the opposite direction: it selects the repositories that are worth keeping an eye on for a future real-time analysis by the described architecture, and then follows them over time, recording which of the selected projects end up with a published CVE and how many days pass between the moment they were selected and that publication. The results it produces are preliminary and are kept separate from the evaluation of the agents, which does not rely on them in any way.

1.3 Thesis Organization

This thesis is structured in seven chapters:

- **Chapter 2 – Background** introduces the concepts the work relies on. It describes how vulnerabilities are disclosed and catalogued, what makes a patch silent, and how large language models are turned into agents that reason and use tools.
- **Chapter 3 – Related Work** reviews the main lines of research on the automated detection of vulnerabilities, including the approaches that rely on language models and agents, and positions the present work with respect to them.
- **Chapter 4 – Methodology** details the data collection and the proposed architecture. It explains how the dataset used for the evaluation was gathered and built, and then presents the three agent configurations, the isolation under which each run is executed and the format in which a verdict has to be returned.
- **Chapter 5 – Repository Monitoring** presents the tool developed as a secondary contribution. It explains how the repositories worth watching are selected, and how the selection is verified over time against the vulnerabilities that are published afterwards.

- **Chapter 6 – Results** reports the experimental results. It defines the evaluation metrics, compares the three configurations and the models under test, and analyses the cases in which the agents failed.
- **Chapter 7 – Conclusions** summarises the findings and discusses directions for future work, including the real time extension built on the monitoring tool.

Bibliography

[1] Black Duck Software. *Open Source Security and Risk Analysis Report*. Annual report. Black Duck Software, Inc., 2026. URL: https://www.blackduck.com/content/dam/black-duck/en-us/reports/rep-ossra.pdf (visited on 08/09/2026) (cit. on pp. 1, 2).

[2] Jessy Ayala, Yu-Jye Tung, and Joshua Garcia. *Investigating Vulnerability Disclosures in Open-Source Software Using Bug Bounty Reports and Security Advisories*. 2025. arXiv: 2501.17748 [cs.CR]. URL: https://arxiv.org/abs/2501.17748 (cit. on pp. 1, 2).

[3] Jiayuan Zhou, Michael Pacheco, Jinfu Chen, Xing Hu, Xin Xia, David Lo, and Ahmed E. Hassan. «CoLeFunDa: Explainable Silent Vulnerability Fix Identification». In: *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. Melbourne, Australia: IEEE, 2023, pp. 2565–2577. DOI: 10.1109/ICSE48619.2023.00214 (cit. on pp. 2, 3).

[4] Nasif Imtiaz, Aniqa Khanom, and Laurie Williams. «Open or Sneaky? Fast or Slow? Light or Heavy?: Investigating Security Releases of Open Source Packages». In: *IEEE Transactions on Software Engineering* 49.4 (Apr. 2023), pp. 1540–1560. DOI: 10.1109/TSE.2022.3181010 (cit. on p. 2).

[5] *GitPatchDB: A Large-Scale GitHub Commit Databank for Vulnerability Patch Analysis*. Under review, International Conference on Learning Representations (ICLR). Revisione a doppio cieco, autori anonimi. 2026 (cit. on p. 3).

[6] Alperen Yildiz, Sin G. Teo, Yiling Lou, Yebo Feng, Chong Wang, and Dinil Mon Divakaran. «Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories». In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. Introduce il benchmark JitVul. Association for Computational Linguistics, 2025. arXiv: 2503.03586. URL: https://aclanthology.org/2025.acl-long.1490/ (cit. on p. 3).

[7] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. «VulDeePecker: A Deep Learning-Based System for Vulnerability Detection». In: *Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS)*. Apre l'era del deep learning per il rilevamento di vulnerabilita' con i code gadget. 2018. arXiv: 1801.01681. URL: https://arxiv.org/abs/1801.01681 (cit. on p. 3).

[8] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. «Deep Learning based Vulnerability Detection: Are We There Yet?» In: *IEEE Transactions on Software Engineering* (2022). Introduce il dataset ReVeal; le prestazioni dei modelli DL calano di oltre il 50% in uno scenario realistico. arXiv: 2009.07235. URL: https://arxiv.org/abs/2009.07235 (cit. on p. 3).

[9] Ze Sheng, Zhicheng Chen, Kewen Zhu, Qingxiao Xu, and Jeff Huang. *Fuzzing-Brain V2: A Multi-Agent LLM System for Automated Vulnerability Discovery and Reproduction*. Finalista DARPA AIxCC; 29 zero-day su 12 progetti open source. 2026. arXiv: 2605.21779 [cs.CR]. URL: https://arxiv.org/abs/2605.21779 (cit. on p. 3).

[10] Nancy Lau, Louis Sloot, Jyoutir Raj, Giuseppe Marco Boscardin, Evan Harris, Dylan Bowman, Mario Brajkovski, and Jaideep Chawla. *ZeroDayBench: Evaluating LLM Agents on Unseen Zero-Day Vulnerabilities for Cyberdefense*. ICLR 2026 Workshop on Agents in the Wild. 2026. arXiv: 2603.02297 [cs.CR]. URL: https://arxiv.org/abs/2603.02297 (cit. on p. 3).